TEIL 1 · OBJEKTORIENTIERTE SOFTWAREENTWICKLUNG

Objekte

Wie stellt man ein Bankkonto im Computer dar? Objekte, Attribute, Zustände und Objektdiagramme

WarnowBank Rostock · Baustein 2 · ca. 15 min

RÜCKBLICK

Wo wir stehen

Die Verb-Substantiv-Methode hat aus der Anforderung zwei Kandidaten geliefert:

Kunde **Konto**

Beide haben **Daten** – Name, Kontostand, PIN ...

Beide haben **Fähigkeiten** – einzahlen, abheben, Name ändern ...

Die Idee der Objektorientierung

Was zusammengehört, wird in einem **Objekt** gebündelt: die Daten und die Fähigkeiten eines Kontos an einer Stelle.

Erinnerung an 0.2: Dort waren Kontostände und Funktionen über das ganze Programm verstreut.

BEGRIFF

Ein Objekt: das Girokonto von Herrn Lehmann

Daten

- Kontonummer 987654321
- PIN 1234
- Kontostand 2.020,00 €
- Dispo 500,00 €

Fähigkeiten

- Geld einzahlen
- Geld abheben
- Geld überweisen
- Kontostand abfragen

Objekt = Daten (**Attribute**) + Fähigkeiten (**Methoden**).
Jedes Objekt hat eine eigene **Identität**: Das Konto von Frau Schulz ist ein anderes Objekt – auch wenn es dieselben Werte hätte.

BEGRIFF

Attribute: Bezeichner und Wert

BEZEICHNER	WERT
`kontonummer`	`987654321`
`pin`	`"1234"`
`kontostand`	`2020.0`
`dispo`	`500.0`

Text steht in Anführungszeichen: `"1234"`

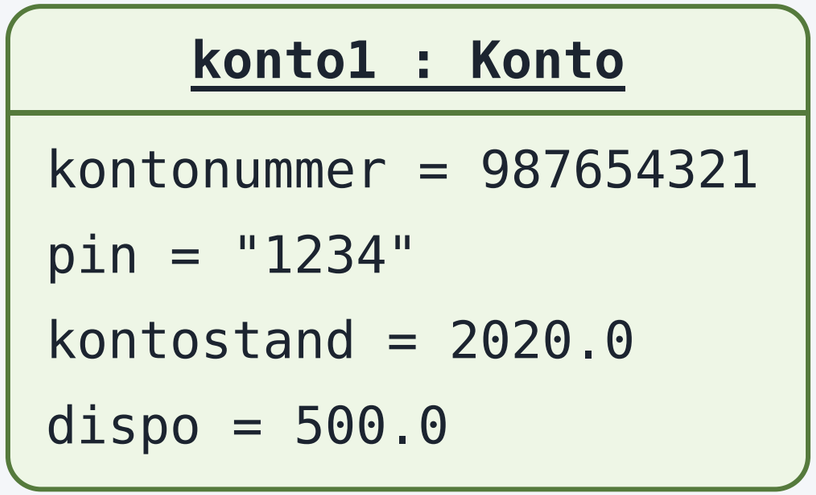
Zahlen ohne Anführungszeichen, Komma als Punkt: `2020.0`

Keine Einheiten: nicht `2020 €`

Bezeichner klein, ohne Umlaute und Leerzeichen

DARSTELLUNG

Das Objektdiagramm (UML)



1. Oben: **Objektbezeichner : Klasse**
2. Kopfzeile **unterstrichen**
3. Rechteck mit **abgerundeten Ecken**
4. Darunter jedes Attribut als **bezeichner = wert**

Die Klasse sagt, *von welcher Art* das Objekt ist. Mehr dazu in Teil 2.

KURZ GEPRÜFT

Text oder Zahl?

Kontonummer
`987654321`

Zahl (`int`): `987654321`

PIN
`0815`

Text (`str`): `"0815"` – als Zahl ginge die Null verloren.

Kontostand
`2.020,50 €`

Zahl (`float`): `2020.5` – Punkt statt Komma, ohne €

Name
Lehmann

Text (`str`): `"Lehmann"`

Geburtsdatum
`14.05.1988`

Text (`str`): `"14.05.1988"` – mehrere Punkte, keine Zahl

Dispo
`500 €`

Zahl (`float`): `500.0`

BEGRIFF

Der Zustand eines Objekts

konto1 : Konto

```
kontonummer = 987654321
pin = "1234"
kontostand = 2020.0
dispo = 500.0
```

Zustand 1

`einzahlen(100)` `abheben(50)` `abheben(3000)`

`von vorn`

Alle Attributwerte zusammen bilden den **Zustand** eines Objekts.
Methoden ändern den Zustand – das Objekt bleibt **dasselbe**.

METHODE

Einen Ablauf als Zustandsfolge darstellen

Beispiel: Herr Lehmann zahlt 100 € ein, danach hebt er 70 € ab.

Zustand 1

konto1 : Konto

kontonummer = 987654321
pin = "1234"
kontostand = 2020.0
dispo = 500.0

einzahlen(100)

→

Zustand 2

konto1 : Konto

kontonummer = 987654321
pin = "1234"
kontostand = 2120.0
dispo = 500.0

abheben(70)

→

Zustand 3

konto1 : Konto

kontonummer = 987654321
pin = "1234"
kontostand = 2050.0
dispo = 500.0

- Nach jedem Vorgang ein neues Diagramm – nur die beteiligten Objekte
- Wird ein Vorgang abgelehnt, bleibt der Zustand unverändert

BEGRIFF

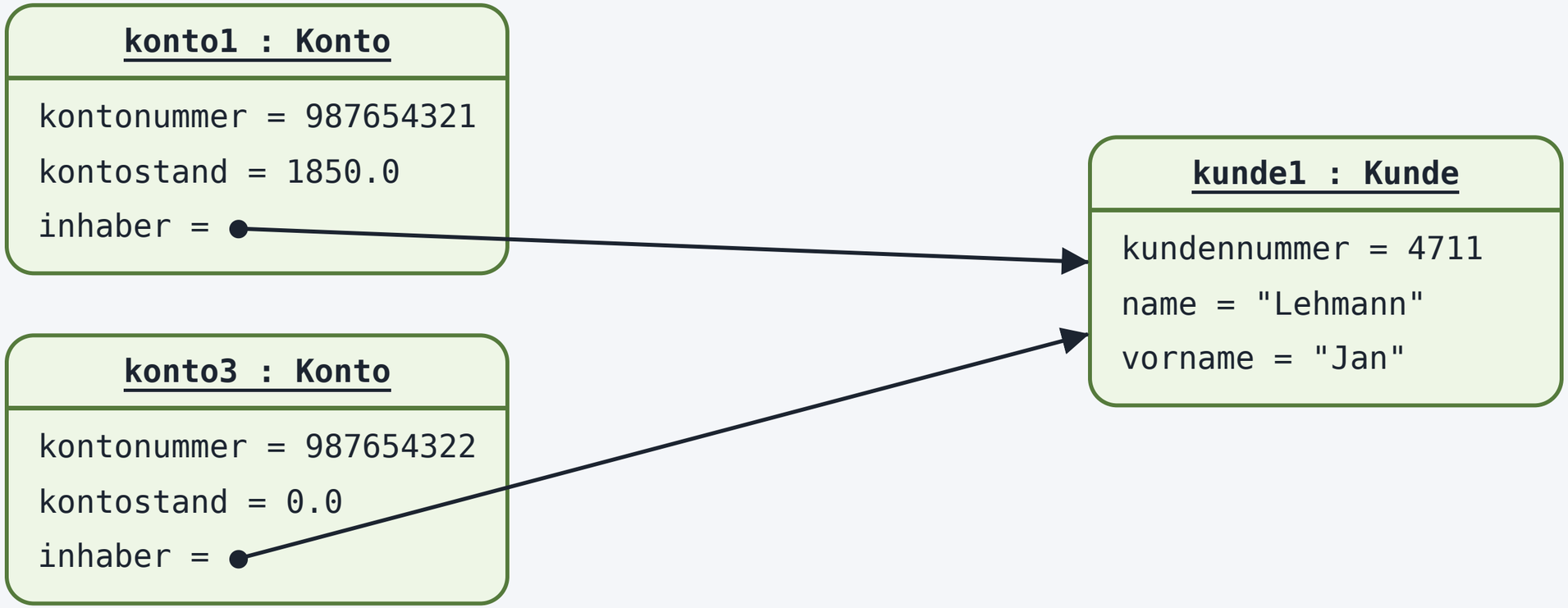
Objekte kennen andere Objekte



- Der Wert von `inhaber` ist **ein anderes Objekt** – gezeichnet als Pfeil: **Objektreferenz**
- Richtung: Das Konto kennt seinen Inhaber. Der Kunde kennt sein Konto (vorerst) nicht.
- Zeigt ein Attribut auf kein Objekt, ist sein Wert `None`.

BEGRIFF

Ein Kunde, zwei Konten



Herr Lehmann heiratet

Es gibt nur **ein** Kundenobjekt. Der Name wird an **einer** Stelle geändert – beide Konten „sehen“ es.

Attribute `pin` und `dispo` hier weggelassen.

INS GLOSSAR

Das Wichtigste aus Teil 1

Objekt	Einheit aus Daten (Attributen) und Fähigkeiten (Methoden) mit eigener Identität und einem Zustand
Attribut	Eigenschaft eines Objekts aus Bezeichner und Wert
Zustand	alle Attributwerte eines Objekts zu einem Zeitpunkt
Objektdiagramm	abgerundetes Rechteck; oben objektbezeichner : Klasse, darunter bezeichner = wert
Objektreferenz	Verweis auf ein anderes Objekt, dargestellt als Pfeil vom Attribut zum Objekt

JETZT SIE

Etappe 2.A: Objekte darstellen

Kursseite, **Baustein 2**

- Aufgabe 2.1 – Ein Objektdiagramm lesen
- Aufgabe 2.2 – Eine zweite Kundin
- Aufgabe 2.3 – Abheben, überweisen, abgelehnt

ca. 30 min (LK 35 min) · danach Stopp

SICHERUNG 2.A

Aufgabe 2.3: die Zustandsfolge

	KONTO1.KONTOSTAND	KONTO2.KONTOSTAND	BEMERKUNG
Zustand 1	2020.0	340.0	
A: 50 € abheben	1970.0	340.0	nur konto1
B: 120 € überweisen	1850.0	460.0	beide Konten
C: 2400 € abheben	1850.0	460.0	abgelehnt

C: 1850 − 2400 = −550 < −500 → Der Dispo würde überschritten. **Die Software muss vor jeder Abhebung prüfen.**

Diese Bedingung programmieren Sie in Baustein 4.

JETZT SIE

Etappe 2.B: Objekte verweisen auf Objekte

- Aufgabe 2.4 – Wem gehört welches Konto?
- Aufgabe 2.5 – Fehler im Objektdiagramm
- Wer fertig ist: Aufgabe 2.6 – Der Geldautomat (LK verbindlich)

ca. 25 min (LK 45 min) · danach Stopp

SICHERUNG 2.B

Aufgabe 2.5: Fehler finden

Konto : konto1
kontonummer = 987654321
pin = 1234
kontostand = "2.020 €"
dispo = 500.0
inhaber = "Jan Lehmann"

Zeile antippen, um den Fehler zu markieren.

1. Objektbezeichner und Klasse vertauscht: `konto1 : Konto`
2. Kopfzeile nicht unterstrichen
3. PIN ist Text: `"1234"`
4. Kontostand ist eine Zahl: `2020.0` – ohne Einheit, ohne Tausenderpunkt
5. `inhaber` ist kein Text, sondern eine Referenz: Pfeil auf `kunde1`

TEIL 2 · OBJEKTORIENTIERTE SOFTWAREENTWICKLUNG

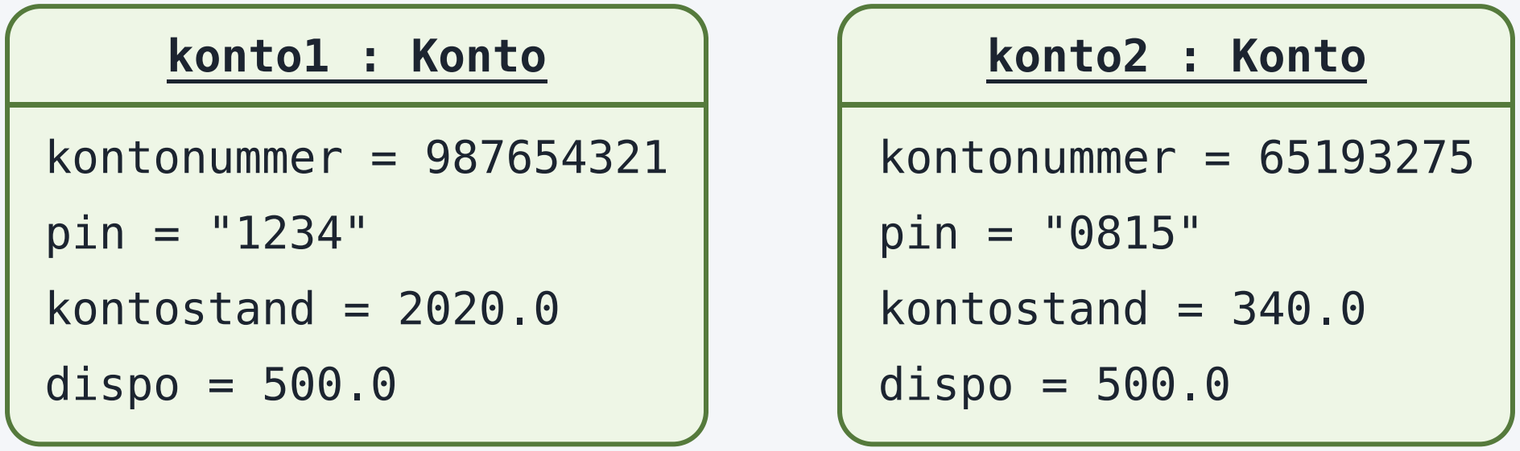
Klassen

Vom einzelnen Objekt zum Bauplan: Entwurfsdiagramme, Datentypen, Methoden und Assoziationen

WarnowBank Rostock · Baustein 3 · ca. 15 min

BEOBACHTUNG

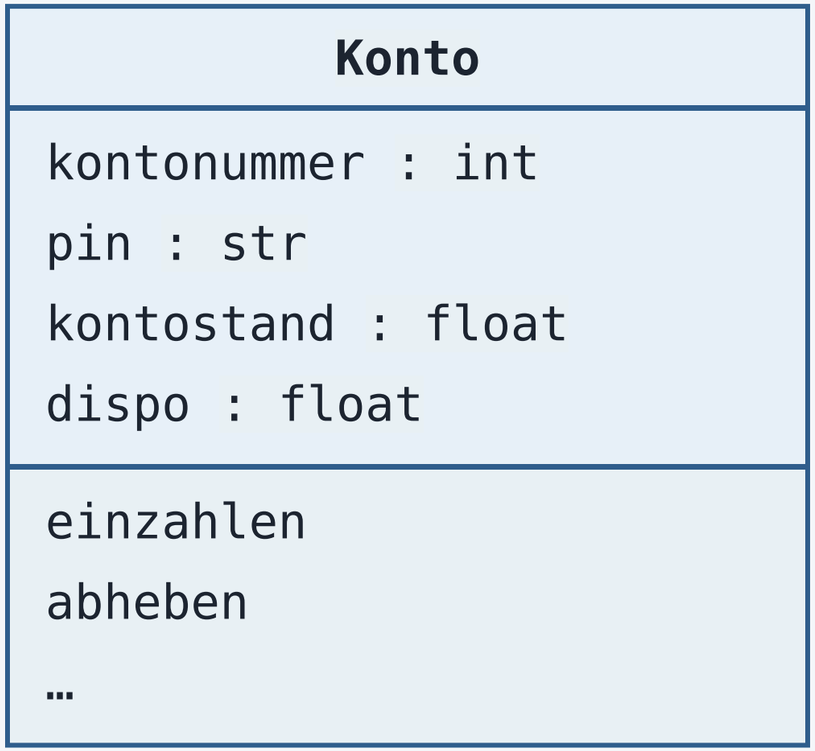
Zwei Konten – was fällt auf?



- **Gleich:** dieselben Attribute, dieselben Fähigkeiten
- **Verschieden:** die Werte – also der Zustand
- Solche Objekte heißen **gleichartig**. Man beschreibt sie gemeinsam in einer **Klasse**.

BEGRIFF

Die Klasse als Bauplan



① **Klassenbezeichner** statt Objektname – groß, nicht unterstrichen

② Statt Werten stehen die **Python-Datentypen**: `int`, `float`, `str`, `bool`

③ Ein dritter Bereich für die **Methoden**

④ Rechteck mit **eckigen** Kanten: das **Entwurfsdiagramm**

Eine **Klasse** ist der Bauplan für gleichartige Objekte. Aus einer Klasse werden beliebig viele Objekte erzeugt.

ENTSCHEIDEN

Datentypen: `int`, `float`, `str` oder `bool`?

Bekannt aus der Programmierung

`int`	`ganze Zahl`	`4711`
`float`	`Kommazahl`	`2020.0`
`str`	`Text`	`"0815"`
`bool`	`wahr oder falsch`	`False`

Prüffragen: Wird gerechnet oder der Größe nach verglichen? → `int` bzw. `float`. Führende 0, Buchstaben, Leer- oder Sonderzeichen? → `str`. Nur ja oder nein? → `bool`

Kartennummer
`4000 1234 5678`

str – Leerzeichen, kein Rechnen

PIN-Fehlversuche
2

int – wird hochgezählt

Karte gesperrt?
`ja / nein`

bool – `True` oder `False`

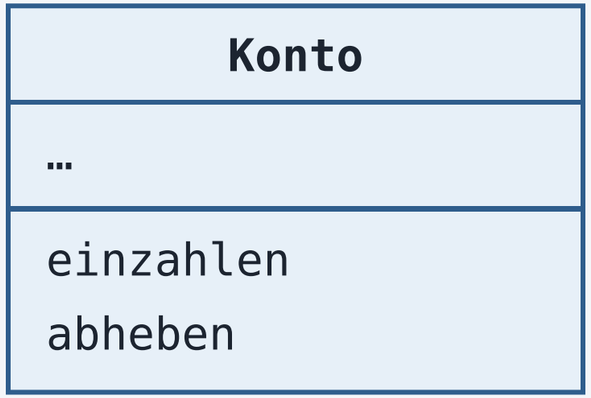
Zinssatz
`1,5 %`

float – `1.5`, damit wird gerechnet

ENTWERFEN

Methoden kommen aus den Verben

Satz 5: „Auf ein Konto kann Geld eingezahlt und von ihm abgehoben werden.“



- Das Verb wird zur **Methode** der Klasse, die die Fähigkeit besitzt
- Konvention: klein, ohne Umlaute, Wortteile mit Unterstrich: `name_aendern`
- Nicht jeder Satz liefert eine Methode – Startwerte beschreiben, was beim **Erzeugen** passiert

KURZ GEPRÜFT

Klasse oder Objekt?

Bankkarte

Klasse – der Bauplan

die Karte mit der Nummer 4000 1234 5678

Objekt – ein bestimmtes Exemplar

Schülerin

Klasse

Lea aus der 11b

Objekt

Stadt

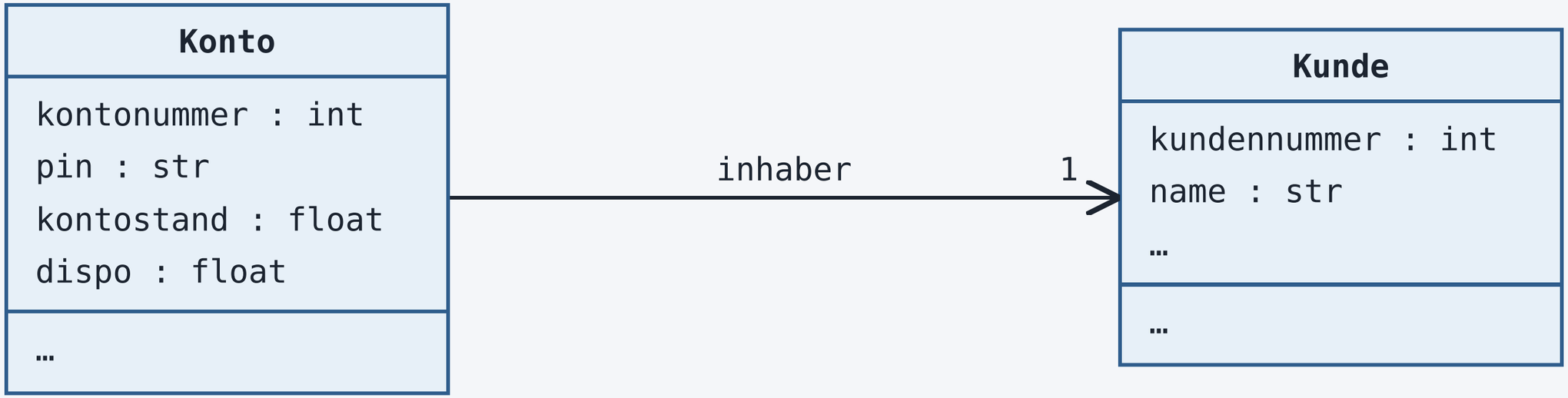
Klasse

Rostock

Objekt – ein Objekt der Klasse Stadt

BEZIEHUNGEN

Aus der Referenz wird eine Assoziation



- Pfeil von Klasse zu Klasse: **Assoziation**
- Am Pfeil der **Bezeichner** des Attributs – nicht zusätzlich im Attributbereich
- An der Spitze die **Multiplizität**: Wie viele Kunden hat ein Konto?

`1`	genau eins	`0..1`	keins oder eins
`1..*`	mindestens eins	`0..*`	beliebig viele

INS GLOSSAR

Das Wichtigste aus Teil 2

Klasse	Bauplan für gleichartige Objekte: dieselben Attribute und Methoden. Attribute mit Datentypen statt Werten.
Entwurfsdiagramm	Rechteck mit drei Bereichen: Klassenbezeichner, Attribute mit Datentyp, Methoden
Assoziation	Beziehung zwischen Klassen über ein Attribut, dessen Datentyp eine andere Klasse ist; Pfeil mit Bezeichner
Multiplizität	wie viele Objekte der anderen Klasse ein Objekt verwalten kann oder muss (1, 0..1, 0..*)

JETZT SIE

Etappe 3.A: Klassen entwerfen

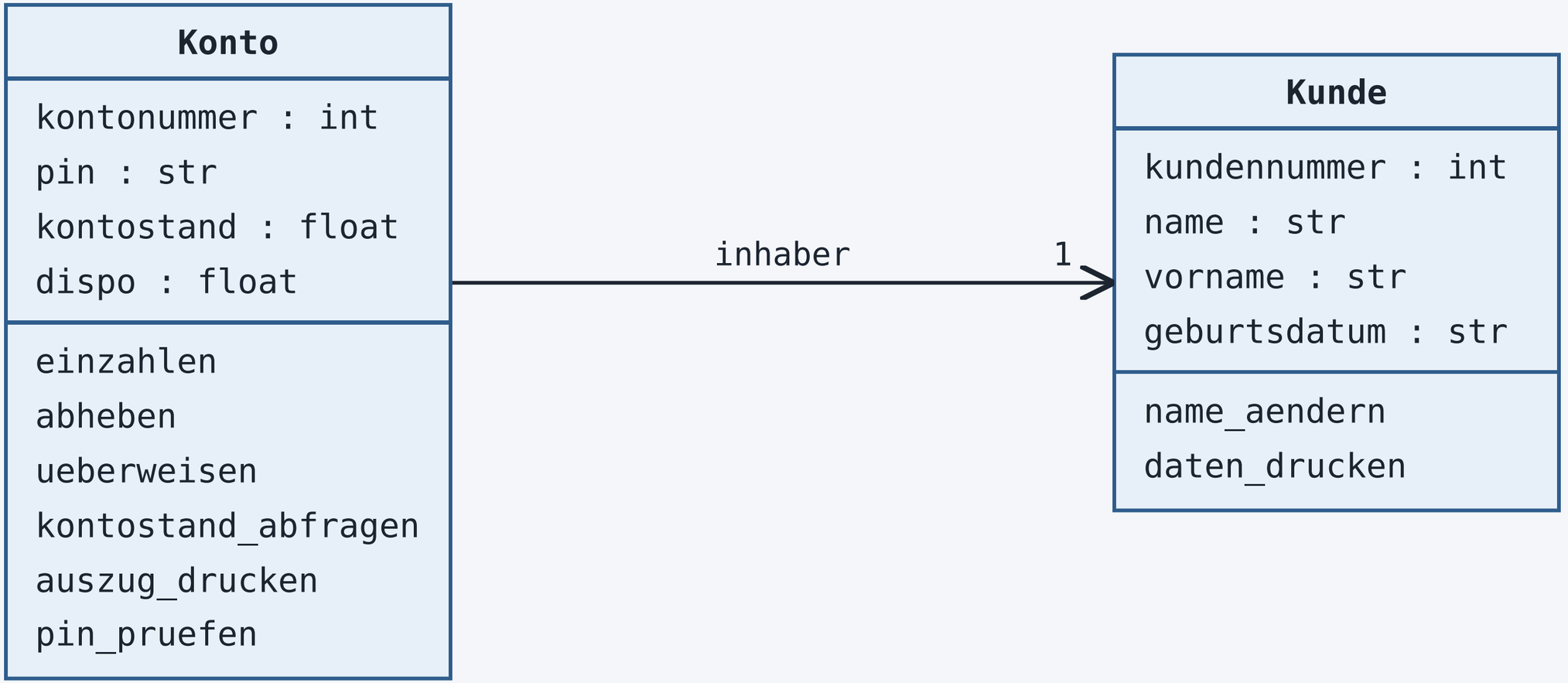
Kursseite, **Baustein 3**

- Aufgabe 3.1 – Kunde und Konto entwerfen
- Aufgabe 3.2 – Klasse oder Objekt?
- Aufgabe 3.3 – Zahl oder Text?
- Aufgabe 3.4 – Die Assoziation interpretieren
- Wer fertig ist: Aufgabe 3.5 – Weitere Klassen der Bank (LK verbindlich)

ca. 30 min (LK 40 min) · danach Stopp

SICHERUNG 3.A

Unser Modell der WarnowBank



TEIL 3 · OBJEKTORIENTIERTE SOFTWAREENTWICKLUNG

Klassen in Python

Vom Entwurfsdiagramm zum Programm: Konstruktor, `self`, Objekte erzeugen und Referenzen

WarnowBank Rostock · Baustein 3 · ca. 15 min · GuiPy bereithalten

ÜBERSETZEN

Vom Entwurfsdiagramm zum Code

Kunde
kundennummer : int name : str vorname : str geburtsdatum : str
name_aendern daten_drucken

Antippen: Was gehört zusammen?

```
class Kunde:
    def __init__(self, kundennummer, name, vorname, geburtsdatum):
        self.kundennummer = kundennummer
        self.name = name
        self.vorname = vorname
        self.geburtsdatum = geburtsdatum
```

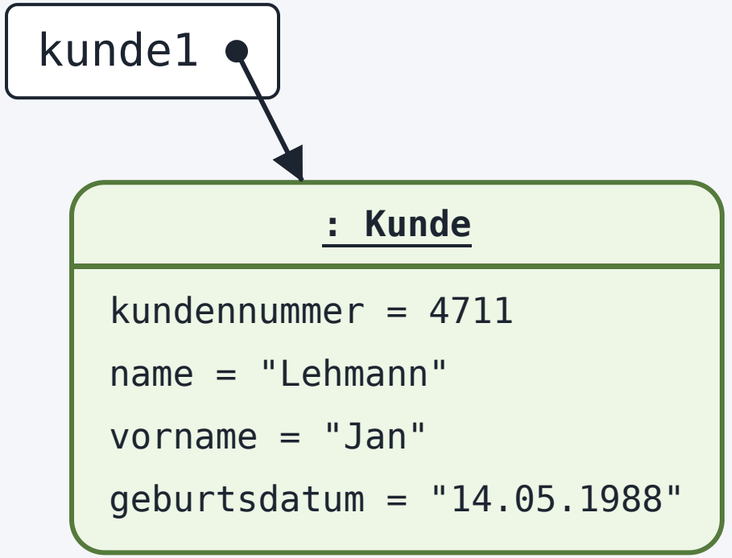
- `class Kunde:` – Klassenname groß, alles Eingerückte gehört dazu
- `__init__` – der **Konstruktor**: legt die Attribute an (je zwei Unterstriche)
- Die Datentypen stehen im Code nicht: Python erkennt sie am Wert – `4711` ist ein `int`
- Methoden folgen in Baustein 4

GENAU HINGESCHAUT

Was passiert beim Erzeugen eines Objekts?

```
class Kunde:
    def __init__(self, kundennummer, name, vorname, geburtsdatum):
        self.kundennummer = kundennummer
        self.name = name
        self.vorname = vorname
        self.geburtsdatum = geburtsdatum

kunde1 = Kunde(4711, "Lehmann", "Jan", "14.05.1988")
```



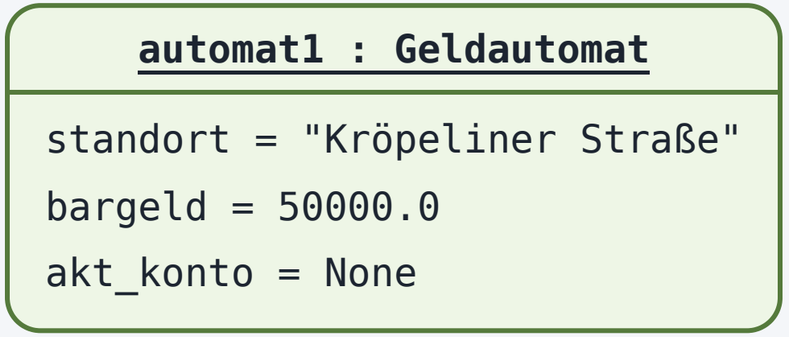
Der Konstruktor endet, die Parameter verschwinden. In `kunde1` wird eine **Referenz** auf das Objekt gespeichert – im Objektdiagramm: kunde1 : Kunde.

KONSTRUKTOR

Nicht jedes Attribut ist ein Parameter

```
class Geldautomat:
    def __init__(self, standort):
        self.standort = standort
        self.bargeld = 50000.0
        self.akt_konto = None

automat1 = Geldautomat("Kröpeliner Straße")
```



- Was für jedes Objekt **anders** ist, wird als Parameter übergeben
- Was für alle neuen Objekte **gleich** ist, setzt der Konstruktor selbst: **fester Startwert**
- `None`: Das Attribut verweist (noch) auf kein Objekt

IM INTERPRETER

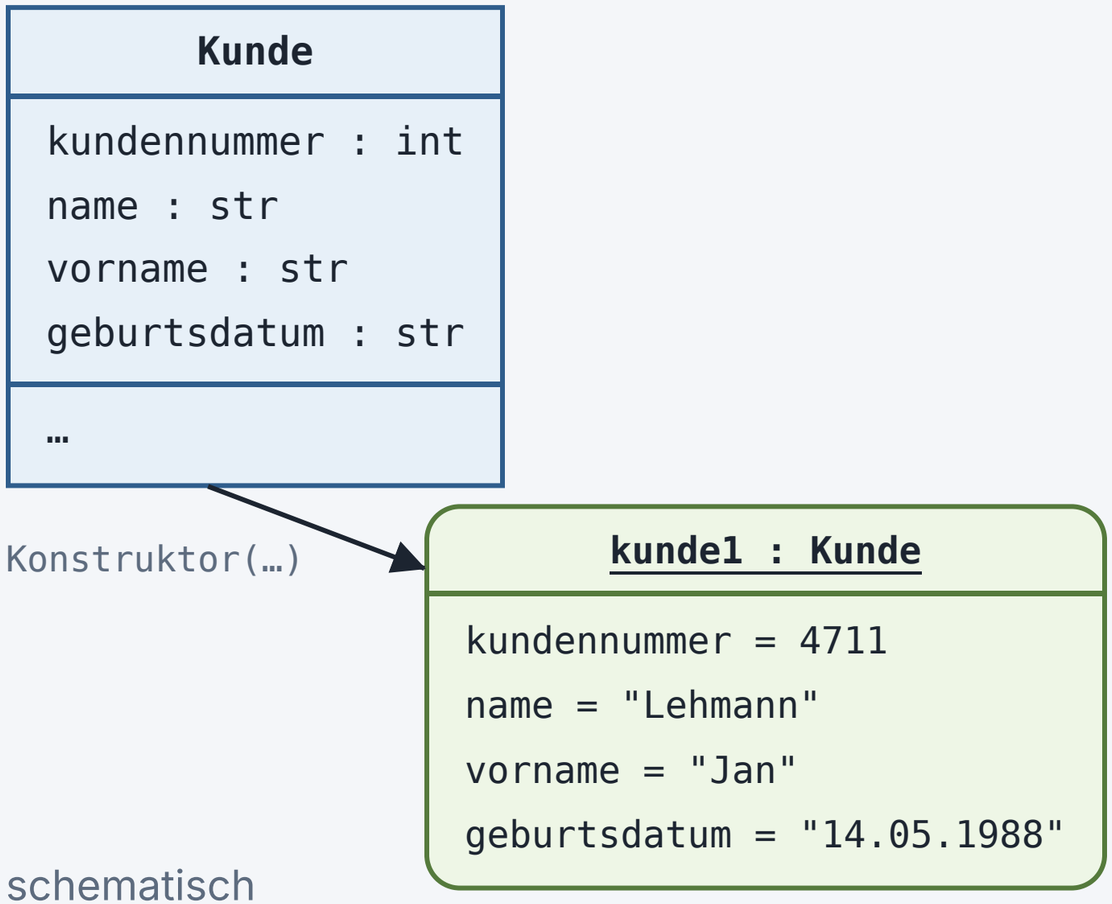
Objekte erzeugen und benutzen

```
# Datei ausführen, dann im Interpreter:
>>> kunde1 = Kunde(4711, "Lehmann", "Jan", "14.05.1988")
>>> kunde1.vorname
'Jan'
>>> kunde1.name = "Lehmann-Krüger"
>>> kunde1.name
'Lehmann-Krüger'
```

- **Instanziierung:** `objekt = Klasse(Werte)` – `self` wird nicht mitgeschrieben
- **Punktnotation:** `objekt.attribut`
- Zuweisung ändert den Zustand (ob das gut ist, klären wir in Baustein 5)

IN GUIPY

Objekte im UML-Fenster



schematisch

1. Datei speichern und ausführen
2. *Diagramm aus geöffneten Dateien*: Die Klassen erscheinen als Entwurfsdiagramm
3. Rechtsklick auf die Klasse → **Konstruktor**, Werte eingeben → das Objekt erscheint mit seinen Attributwerten
4. Rechtsklick auf das Objekt: Attribute bearbeiten, Methoden aufrufen – danach wird der neue Zustand angezeigt

Code geändert? Die Klasse wird **violett** → neu ausführen. Die Objekte gehen dabei verloren.

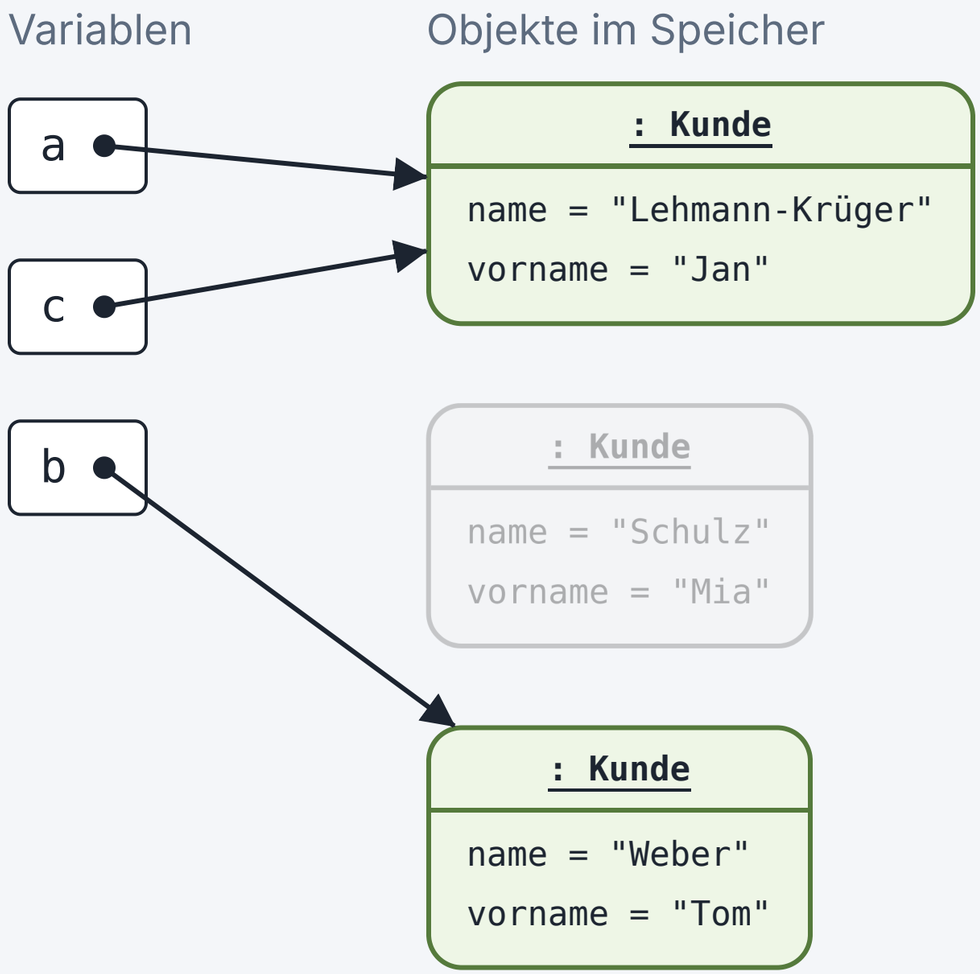
GENAU HINGESCHAUT

Variablen speichern Referenzen

```
a = Kunde(4711, "Lehmann", "Jan", "14.05.1988")
b = Kunde(4712, "Schulz", "Mia", "02.11.2001")
c = a
c.name = "Lehmann-Krüger"
b = Kunde(4713, "Weber", "Tom", "30.01.1999")
```

b verweist auf ein neues Objekt. Auf Frau Schulz zeigt nichts mehr – das Objekt ist nicht mehr erreichbar und wird von Python gelöscht.

Schritt 6 / 6



WENN ES NICHT LÄUFT

Fehlermeldungen lesen

`NameError: name 'Kunde' is not defined`

Die Datei wurde nicht ausgeführt – oder der Klassenname ist falsch geschrieben (`kunde` statt `Kunde`).

`IndentationError: expected an indented block after function definition`

Der Rumpf von `__init__` ist nicht eingerückt. Alles, was zum Konstruktor gehört, steht eine Stufe weiter rechts.

`TypeError: Kunde.__init__() takes 5 positional arguments but 6 were given`

Beim Erzeugen wurde ein Wert zu viel übergeben. Python zählt `self` mit: 4 Werte + `self` = 5.

`AttributeError: 'Kunde' object has no attribute 'nachname'`

Das Attribut heißt anders: Im Konstruktor wurde `self.name` angelegt, nicht `self.nachname`.

Faustregel: Die letzte Zeile der Meldung wörtlich übersetzen – sie verrät fast immer die Ursache.

INS GLOSSAR

Das Wichtigste aus Teil 3

Konstruktor	Methode `__init__`, die beim Erzeugen automatisch ausgeführt wird; legt die Attribute an und belegt sie mit Parametern oder festen Startwerten
`self`	das Objekt, das gerade erzeugt bzw. benutzt wird; immer erster Parameter
Instanziierung	Erzeugen eines Objekts: `objektname = Klassenname(Werte)`
Referenz	Variablen speichern Verweise auf Objekte; `c = a` kopiert den Verweis, nicht das Objekt

JETZT SIE

Etappe 3.B: Klassen in Python

- Aufgabe 3.6 – Kunde und Konto implementieren (in GuiPy, neue Datei `bank.py`)
- Aufgabe 3.7 – Zwei gleiche Kunden?
- Aufgabe 3.8 – Wie viele Konten gibt es?
- Aufgabe 3.9 – Fehlersuche
- Wer fertig ist: 3.10 Die Klasse Karte, 3.11 Das Konto kennt seinen Inhaber (LK verbindlich)

ca. 40 min (LK 55 min) · danach Stopp

SICHERUNG 3.B

Unser Stand von `bank.py`

```python
class Kunde:
    def __init__(self, kundennummer, name, vorname, geburtsdatum):
        self.kundennummer = kundennummer
        self.name = name
        self.vorname = vorname
        self.geburtsdatum = geburtsdatum

class Konto:
    def __init__(self, kontonummer, pin):
        self.kontonummer = kontonummer
        self.pin = pin
        self.kontostand = 0.0
        self.dispo = 500.0
```

Danach erhalten alle diesen Stand über den Austauschordner.

Kurz-Check

Unterschied: `Konto`, `konto1`, `konto1.kontostand`

Klasse (Bauplan) – Objekt (Variable mit Referenz) – Attribut des Objekts

Aufgabe des Konstruktors

legt beim Erzeugen die Attribute an und belegt sie mit Startwerten

„falsch“ in Python

`False` – großgeschrieben, ohne Anführungszeichen